CUSTOMIZATION BUILDER

Table of Contents

Basic Integration 2
Builder API 4
Methods 21
Contacting the Team 33
Document Change Log 33
Next Steps 34

Customization Builder

Last Updated: 04/24/2020

This is the official reference for the features and functionality of the Customization Experience Builder, a product in the **Customization Experience Platform (CXP) (https://nde-devportal-docs.niketech.com/doc/commerce/customization/overview-customization.html)**.

> **TIP**: Also see **Customization Overview (https://nde-devportal-docs.niketech.com/doc/commerce/customization/overview-customization.html)** and **Adding Customization To Your Experience (https://nde-devportal-docs.niketech.com/doc/commerce/customization/use-customization/html)**.

The Builder is a JavaScript bundle that is your main interface with CXP. It offers the following:

- **UX**: Returns a fully-styled UX for customizing products
- **Data API**: Interact with **Build Data** and CXP REST APIs

Basic Integration

This section describes how to complete a basic integration of the Builder into a web view or browser-based application.

> **TIP**: For more detailed integration instructions see **Adding Customization To Your Experience (https://nde-devportal-docs.niketech.com/doc/commerce/customization/use-customization.html)**.

1. Include the Builder Bundle

In your app's source (e.g. in an HTML template), include the Builder JavaScript bundle in a `<script>` tag in the `<body>` like:

```
  <script src="https://assets.commerce.nikecloud.com/nikeid/builder/dist/
b16Builder.bundle.min.js" type="text/javascript"></script>
```

This loads the Builder bundle when the page is rendered.

Bundle URIs

The bundle URIs to be used in the Test and Production environments are provided below. Reach out to the CXP team to determine which version you should be using.

Test

- Builder UI: https://assets.test.commerce.nikecloud.com/nikeid/builder/dist/b16Builder.[bundleVersion].min.js
- Product API: https://assets.test.commerce.nikecloud.com/nikeid/builder/dist/b16ProductApi.[bundleVersion].min.js

Production

- Builder UI: https://assets.commerce.nikecloud.com/nikeid/builder/dist/b16Builder.[bundleVersion].min.js
- Product API: https://assets.commerce.nikecloud.com/nikeid/builder/dist/b16ProductApi.[bundleVersion].min.js

2. Add a for the Builder to Load Into

Add a `<div>` in the `<body>` with an `id="nikeid-app"` attribute like:

```
<div id="nikeid-app-container" style="width: 1000px; height: 800px;">
  <div id="nikeid-app">
  </div>
</div>
```

You can name the `id` attribute however you want, just be sure to use the same name in the next step.

3. Initialize the Builder

Initialize the Builder by invoking the `nikeIdBuilder(rootElement, config)` function in a `<script>` tag in the `<body>` like:

```
<script>
 const builderApi = nikeIdBuilder(document.getElementById('nikeid-app'), {
   'nike-api-caller-id': 'com.nike:commerce.b16.localhost',
   pathName: 'KobeAD2exoFA18'
 })
</script>
```

This returns the **Builder API** and renders the Customization UX into the specified HTMLElement.

> **TIPS**:
>
> - Make sure to use the same `id` used in Step 2 in the `document.getElementById()` argument.
> - The `nikeIdBuilder` function is described in detail in **Builder API**

4. Navigate to the URI to Launch the Experience

- Serve your app/page locally by running `npm start` or `yarn start`.
- Launch the Customization experience by navigating to the URI with at least the `?pathName` query parameter. The value in `?pathName` must match what you set in the `pathName` property when the Builder was initialized, like:

```
http://localhost:3000/?pathName=KobeAD2exoFA18
```

Additional Query Params

The following optional query parameters can be used in the URI to affect how the Customization experience is

rendered.

- `imageSize`: sets the size of images returned by the Builder in pixels (max 666)
- `imageQuality`: sets the quality (as bit-depth) of the PNG images returned by the Builder (8/24)
- `hideMenu`: shows/hides the harnessMenu on page load (true/false)

Example:

```
 http://localhost:3000/?imageSize=666&imageQuality=8&hideMenu=true&pathName=metcon2016160
1
```

Integration Flow

- Invoke `nikeIdBuilder(rootElement, config)` to get the Builder API. The returned object also contains a method for api.onApiReady, which indicates that the Builder has loaded.
- Listen to price change, analytic, and “done” events coming from the Builder.
- For buying tools, use `setSizeType` and `setSizeAnswer` for answering the size-related questions. Use `setAnswer` for answering Gender and Width questions.
- Invoke `setBuild` for loading a new build by prebuild id, metric id, or raw build data. This is mainly meant to “reset” the Builder.

Builder API

Invoking the `nikeIdBuilder(rootElement, config)` function returns the Builder API.

Usage:

```
 /**
 * Function nikeIdBuilder returns the Builder API
 * @param {HTMLElement} rootElement The DOM element in which to inject the Builder
 * @param {Object} config The configuration properties that identify which Builder
instance to be returned
 * @return {Object} Returns the exposed builderApi handlers
*/
var rootElement = document.getElementById('my-app-location');
var config = {
  'nike-api-caller-id': 'XXXXX',
  locale: 'en_US',
  country: 'US',
  pathName: 'af1High14_July',
  bridge: {
    // ...
  }
  // ... etc, documented below
};
```

```
var builderApi = nikeIdBuilder(rootElement, config)
```

Parameters

rootElement

The argument for the `rootElement` parameter must be an HTMLElement target to mount the Builder application into.

Example: `document.getElementById('nike-id-builder-app');`

config

The argument for the `config` parameter must include the configuration properties that identify which Builder instance to be returned.

At minimum, the `pathName` and `nike-api-caller-id` properties are required in order to initialize the Builder.

```
{
 // required properties
 'nike-api-caller-id': 'com.nike:test',
 pathName: 'af1High14_July',

 // sometimes required properties
 productId: 'PROD359583',
 country: 'US',
 locale: 'en_US',

 // suggested bridge
 bridge: {
   onAnalyticsEvent: function() {},
   onApiReady: function(api) {}, // api ready not build
   onDone: function(buildData) {},
   onError: function(error){},
   onPriceUpdate: function(priceData) {},
   onProductLoad: function(buildData) {} // build ready
 }

 // optional properties
 abTestCookieName: null,
 builderMode: 'default',
 currentEnv: 'PROD',
 dataAsStrings: false,
```

```
  disablePalette: false,
  eMerchId: 123456,
  enableAnimationInit: false,
  fetchCapacity: false,
  fitToContainer: document.getElementById('nikeid-app-container'),
  hasTouch: true,
  isColorMatching: false,
  metricId: '42',
  prebuildId: '42',
  sizeTypeRegion: 'US',
  myDesignsEnabled: true,
  myDesignCapacity: 20,

  // optional skip price info
  skipPriceInfo: false,
  colorCode: '994',
  styleCode:'AA1617',

  // optional image quality
  imageQuality: 24,
  imageSize: 300,
  viewPortSize: 'S',
}
```

Required Properties

Table 1: Required Build Properties

`pathName`	String	The product pathName for the requested build.
`nike-api-caller-id`	String	A platform-unique key («domain name»:«appid») that identifies the API caller to customization services. See **Architecture Standards (https://github.nike.com/ea-governance/ea-standards/blob/master/api-standards/api-standards-main/API_Standards.md#identifying-a-calling-client)** for more.

Getting a `nike-api-caller-id`

Please contact a team member in **#nikeid-dev-systems (https://nikedigital.slack.com/messages/C0L8C4UM7)** or email **Lst-digitaltech.customization.id.systems@nike.com** to obtain a key for your platform in advance of making API calls. The current list of supported values is shown below (Note: items marked with * are for internal tracking purposes):

Table 2: Nike Platforms with Corresponding Caller IDs

Platform	Value
Nike.com Desktop PDP	com.nike:commerce.idpdp.desktop
Nike.com Mobile PDP	com.nike:commerce.idpdp.mobile
Nike.com Jersey PDP	com.nike:commerce.jerseyidpdp.desktop
Nike.com Jersey Mobile PDP	com.nike:commerce.jerseyidpdp.mobile
Nike.com Desktop 3D Builder	com.nike:commerce.3d.builder.desktop
Nike.com Mobile 3D Builder	com.nike:commerce.3d.builder.mobile
Converse.com Desktop PDP	com.converse:commerce.idpdp.desktop
Converse.com Mobile PDP	com.converse:commerce.idpdp.mobile
Converse.com Europe Desktop PDP	com.converse.eu:commerce.idpdp.desktop
Converse.com Europe Mobile PDP	com.converse.eu:commerce.idpdp.mobile
Nike App iOS	com.nike.commerce.omega.ios
Nike App Android	com.nike.commerce.omega.droid
NikeElite.com Desktop PDP	com.nike.elite:commerce.idpdp.desktop
NikeElite.com Mobile PDP	com.nike.elite:commerce.idpdp.mobile
Nike Cultivator Desktop	com.nike.cultivator:commerce.desktop
Nike Cultivator Mobile	com.nike.cultivator:commerce.mobile
Nike Factory Collab (FC) (includes FC Lite)	com.nike.commerce.fc
Nike DOMS	com.nike.commerce.doms
Nike CSP	com.nike.commerce.csp
Nike Order Capture (OCP)	com.nike.commerce.ocp
Nike PI	com.nike.commerce.pi

Platform	Value
Nike Email	com.nike.commerce.email
Tmall.com (Desktop)	com.tmall:commerce.idpdp.desktop
Tmall.com (Mobile)	com.tmall:commerce.idpdp.mobile
WIP Tool*	com.nike:wip
Test*	com.nike:test
Localhost*	com.nike:commerce.b16.localhost

Properties With Dependencies

Table 3: Build Properties Having Dependencies

`productId`	String	Product ID for the requested build. Only required when using b16Builder and passing config: builderMode: ‘wip’
`country`	String	The current country two character abbreviation, e.g. `US`, `GB`, `CN`. Only required when using builderProductApi
`locale`	String	The current locale abbreviation, e.g. `en_US`, `en_GB`, `zn_CN`. Only required when using builderProductApi

Optional Properties

Table 4: Optional Build Properties

`abTestCookieName`	String	A unique user id for Optimizely to use for it’s a/b tests. If `anonymousId` cookie is present, it will be used over the abTestCookieName.
`builderMode`	String	Determines which services are used by the Builder for product data and scene7 calls. In `default` mode, the production app uses a

		service call to the V4 Product service in production. In `wip` mode, the Builder uses the idedit preview product service and you must also supply the `productId` to be passed along in the service call.
`currentEnv`	String	Determines the environment to be used for Builder backend services. Possible values: `PROD`, `ecnXX` (any ecn environment), `prdvtools1` (any preview environment). If this config value is not set, the Builder will attempt to derive it from the client host URL, and if unsuccessful will default to `PROD`.
`dataAsStrings`	Boolean	Pass `true` to have the Builder API methods return data as a string, avoiding unwanted marshaling side effects.
`disablePalette`	Boolean	Pass `true` to disable/hide the marketing components/palette navigation.
`eMerchId`	String	The prodigy product ID. This can be used to fetch VAS SKU data required for jerseys.
`enableAnimationInit`	Boolean	Pass `true` if you want to await palette item animations until `builderApi.setIsVisible(true)` is called on the Builder
`fetchCapacity`	Boolean	Pass `true` to fetch capacity information along with the product.
`fitToContainer`	HTML Element	When set to a DOM element (e.g document.getElementById('container')), the experience will only flex to the size of that container rather than attempting to use 100vh or calculating UI dimensions based

		on the window.
`hasTouch`	Boolean	Pass `true` if the experience supports touch input.
`isColorMatching`	Boolean	Passing `true` hides the harness menu and control buttons. This supports color matching work flow in the WIP Builder.
`metricId`	String	Passing a `prebuildId` or `metricId` in config will force the Builder to load the build for the product and apply all the selections in the build to the loaded product. `metricId` will always be preferred over `prebuildId` if both are supplied.
`prebuildId`	String	Passing a `prebuildId` or `metricId` in config will force the Builder to load the build for the product and apply all the selections in the build to the loaded product. `metricId` will always be preferred over `prebuildId` if both are supplied.
`sizeTypeRegion`	String	The sizeTypeRegion corresponding to the list of sizeTypes that should be returned in the build data. Valid values are `US`, `EU`, `JP`, `CN`, `XP`.
`viewPortSize`	String	Sets viewport size, and creates values for `imageSize` and `imageQuality`. Can be `'S'`, `'M'`, `'L'`, `'XL'`, `'S8'`, `'M8'`, `'L8'`, `'XL8'`.
`imageQuality`	Number	Sets the PNG bit depth. Can be either `8` or `24`.
`imageSize`	Number	Number of pixels at which the main shoe image will be rendered.

`skipPriceInfo`	Boolean	If set to `true`, the pricing information won't be returned but style-color code will still be returned. Use for any product that is not setup in Prodigy. This flag is added to support Converse EU products.
`styleCode`	String	Mandatory when skipPriceInfo is set to `true`. Causes the build to be saved with the styleCode from the input config.
`colorCode`	String	Mandatory when skipPriceInfo is set to `true`. Causes the build to be saved with the colorCode from the input config.
`myDesignsEnabled`	Boolean	If set to 'true', My Designs local storage will be enabled. Default is 'false'
`myDesignCapacity`	Number	Number of most recent designs to be stored in local storage. If this maximum value is exceeded, the oldest design(s) will be deleted from storage. Default is '15'

Bridge Properties

The bridge is a set of callbacks provided to the Builder config under the `config.bridge` property. They allow you to be notified by actions within the Builder.

onAnalyticsEvent(type, payload)

Called upon user invoked events for capturing analytics data.

The `type` property will be an event name that is exposed for analytics. This list includes:

- **ANALYTICS_APPLY_CUSTOMIZATION**: When the user makes any customization change
- **ANALYTICS_COMPLETE_BUILD**: When the user presses 'done' to complete the build
- **ANALYTICS_MARKETING_COMPONENT_SELECTED**: When the user selects a marketing component
- **ANALYTICS_SELECT_VIEW**: When the user selects a view from the carousel
- **ANALYTICS_ZOOM_IN**: When the user requests a higher-res image

The `payload` property contains meta information about the event type. For example, the meta information for

`ANALYTICS_SELECT_VIEW` would include the selected view number. The same applies for `ANALYTICS_MARKETING_COMPONENT_SELECTED`, which will include the id and name of the marketing component.

onApiReady(api)

Called when the API is ready, shortly after the product has loaded. This returns the API object.

> **NOTE**: The build is not always ready at this point. Please use `bridge.onProductLoad(buildData)` for initial build. Then use `api.getBuild()` after that.

onDone(buildData)

Called with the current buildData whenever is “done” editing their build.

onError(error)

Called when an error response is returned from product data fetching or when a initialization error occurs on the client. Error may be either a string or a native JavaScript error.

onPriceUpdate(priceData)

Called with price information whenever the user makes selection changes.

onProductLoad(buildData)

Called with the current buildData state of the Builder whenever a new product is loaded, or a build is applied.

Build Data

The Builder returns a `buildData` object (also referred to as Build Data), which describes the current state of the build with sizing information, style/color, current product question/answer pairs, and more. Here is a **sample buildData object (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**.

Build Data Fields

Table 5: Fields in Build Data

Field	Type	Description	Example
`pathName`	String	The product pathName for this build.	“ER2teamSP19_barca”
`productId`	String	The product ID for this build.	“PROD372041”
`consumerQuesAnswers`	Array	Collection of product question-and-answer pairs representing the	See **here (https://nde-devportal-docs.niketech.com/**

Field	Type	Description	Example
		current state of the Builder's pid selections.	doc/commerce/customization/buildDataExample.html)
`productQuesAnswers`	Array	A collection of product question-and-answer pairs representing the current state of the Builder's combination selections.	See here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)
`sizeMarketingComponent`	Object	DEPRECATED and replaced by `sizingData` (see below)	
`viewNumbers`	String	Comma-separated list of views currently being displayed by the Builder, in the order in which they are being displayed. Provides a way to template scene7 URLs for each angle of the product currently being shown within the Builder for your own product carousels or display pages.	"1,2,3,4,5,6"
`color`	String	The product color code for this build.	"994"
`price`	String	The product price formatted as string with the currency symbol.	$180
`rawPrice`	Number	The product price formatted as a number.	180
`style`	String	The product style code.	"CK3977"

Field	Type	Description	Example
`country`	String	The current country code for this build.	“US”
`locale`	String	The current locale code for this build.	“en_US”
`imageNumbers`	String	Comma-separated list of views currently being displayed by the Builder, in the order in which they are being displayed. Provides a way to template scene7 URLs for each angle of the product for your own product carousels or display pages.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**
`imageService`	String	The host of the service call used to request images of the product represented by the buildData. Use in conjunction with the viewUrlTemplate field to build scene7 image service request URLs.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**
`imageUrlTemplate`	String	Scene7 parameterized URL for making requests for product images that are returned as opaque JPGs. The URL contains tags that must be replaced with valid values. These tags are {VIEW_NUMBER} (one of the values from imageNumbers), {IMAGE_WIDTH}, {BACKGROUND_COLO	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**

Field	Type	Description	Example
		R} (in the form f5f5f5), & {JPG_QUALITY} (0 - 100).	
`imageUrlShadowTemplate`	String	This URL is the same as the imageUrlTemplate except it has added parameters that will cause a shadow to be rendered underneath the image.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**
`viewService`	String	The host of the render service call used to request images of the product represented by the buildData. Use in conjunction with the viewUrlTemplate field to build scene7 render service request URL.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**
`viewUrlTemplate`	String	Scene7 parameterized URL for making requests for product images that are uncompressed PNGs with a transparent background. The URL contains a tag {VIEW_NUMBER} (one of the values from imageNumbers) that must be replaced with a valid value. You must also add a width to the end of the string in the form &wid={imageWidth} where {imageWidth} represents the width you desire.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**

Field	Type	Description	Example
`viewUrlShadowTemplate`	String	This URL is the same as the viewUrlTemplate except it has added parameters that will cause a shadow to be rendered underneath the image.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**
`availability`	Object	The lead time in days for the product to be delivered, when `isAvailable` is true. When false, only an 'out of stock' message is returned.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**
`myDesigns`	Array	Array of locally-saved My Designs for the given pathName	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**
`sizeTypes`	Array	A collection of sizeTypes based on the sizeTypeRegion that was requested in the config data.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**
`sizingData`	Array	Contains arrays of gender, width, size, and size chart data. Used for "buying" and "size" tools.	See **here (https://nde-devportal-docs.niketech.com/doc/commerce/customization/buildDataExample.html)**

More about **availability**

The below table describes the contents of the `availability` object:

Table 6: Fields in Availability Object

Field	Type	Description	Example
`isAvailable`	Boolean	Whether the current style-color is available to be purchased	true
`leadtimeUpperBoundInDays`	Number	The lead time for the product to be delivered, in days	22
`longCapacityMessage`	String	The long message text describing the product availability and lead-time.	“Custom-made and delivered to you in 3 weeks or less.”
`shortCapacityMessage`	String	The short message text describing the product availability and lead-time.	“GREAT CHOICE”

More about **sizingData**

- Render the gender, width, and size questions and answer them using the `setAnswer` and `setsizeAnswer` methods as the consumer makes selections.
- If a value for the `sizeTypeRegion` field is not provided when initializing the Builder, then `sizeTypes` will default to undefined.
- If `fetchCapacity` is set to false when initializing the Builder, then all sizes will show as out of stock.
- Capacity, inventory, restrictions, sizeType and prior answers to sizing questions are all taken into consideration when generating this collection of objects.

Sample sizingData:


```
[
 {
   "displayName":"Size Chart",
   "questionId":"huarRunESS1702:LTITEM8538:LTITEM8112:LTITEM8010:LTITEM380033",
   "type":"SzChart",
   "answers":
   [
     {
       "path":"/v2/sizecharts/men_footwear_default.xml",
       "answerId":"LTITEM10027",
```

```
        "type":"SzChart",
        "isDefualt":true
      }
    ]
  },
  {
    "displayName":"Gender",
    "questionId":"huarRunESS1702:LTITEM8538:LTITEM8112",
    "type":"Gender",
    "answers":
    [
      {
        "displayName":"Mens",
        "questionId":"huarRunESS1702:LTITEM8538:LTITEM8112",
        "answerId":"LTITEM8010",
        "type":"Gender",
        "isDefault":true,
        "isSelected":false,
        "isRestricted":false
      },
      {
        "displayName":"Womens",
        "questionId":"huarRunESS1702:LTITEM8538:LTITEM8112",
        "answerId":"LTITEM8011",
        "type":"Gender",
        "isDefault":false,
        "isSelected":false,
        "isRestricted":false
      }
    ]
  },
  {
    "displayName":"Width",
    "questionId":"huarRunESS1702:LTITEM8538:LTITEM210042",
    "type":"Width",
    "answers":
    [
      {
        "displayName":"Narrow Fit",
        "questionId":"huarRunESS1702:LTITEM8538:LTITEM210042",
        "answerId":"LTITEM8098",
        "type":"Width",
```

```
        "isDefault":false,
        "isSelected":false,
        "isRestricted":false
      }
    ]
  },
  {
    "displayName":"Size",
    "questionId":"huarRunESS1702:LTITEM8538:LTITEM8112:LTITEM8010:LTITEM380033",
    "importantInformation":"This product runs about a half-size smaller than standard
Nike sizing.",
    "type":"Sz",
    "sizetype":"euro",
    "answers":
    [
      {
        "displayName":"39",
        "questionId":"huarRunESS1702:LTITEM8538:LTITEM8112:LTITEM8010:LTITEM380033",
        "answerId":"LTITEM8128",
        "type":"Sz",
        "sizetype":"euro",
        "isDefault":false,
        "isSelected":false,
        "isAvailable": true,
        "constrains": {
          "isInStock":true,
          "hasCapacity":true,
          "isRestricted":false
        }
      }
    ]
  }
]
```


Working with imageServer Images

The buildData that is returned includes URLs for generating images from the **image server**. These images are opaque JPGs, as opposed to the transparent PNGs provided by the **render server**.

- `imageNumbers`
- `imageService`
- `imageUrlTemplate`
- `imageUrlShadowTemplate`

These can be processed to generate the urls you need by adding this function to your code:

```
 /**
* Get the imageUrls for the current design from the image server
*
* @param {Object} buildData - the buildData
* @param {String} backgroundColor - the background color to use i.e. ('f5f5f5')
* @param {Integer} imageWidth - the width value of the image
* @param {Integer} jpgQuality - the jpeg compression quality (0 - 100), I recommend 90
* @param {Boolean} includeShadow - whether or not to include shadow
* @returns {Array} - an array of urls
*/
getImageUrls(buildData, backgroundColor, imageWidth, jpgQuality, includeShadow) {
    const imageUrls = [];
    const imageService = buildData.imageService;
    const imageTemplate = (includeShadow ? buildData.imageUrlShadowTemplate :
buildData.imageUrlTemplate);

    if (buildData && buildData.imageNumbers && imageService && imageTemplate) {
        const viewNumbers = buildData.imageNumbers.split(',');
        const baseViewUrl = `${imageService}${imageTemplate}`;

        for (let viewIndex = 0; viewIndex < viewNumbers.length; viewIndex += 1) {
            let imageUrl = baseViewUrl.replace(/{VIEW_NUMBER}/, viewNumbers[viewIndex])
            .replace(/{IMAGE_WIDTH}/, imageWidth)
            .replace(/{BACKGROUND_COLOR}/, backgroundColor)
            .replace(/{JPG_QUALITY}/, jpgQuality);

            imageUrls.push(imageUrl);
        }
    }

    return imageUrls;
}
```

Working with viewServer Images

The buildData that is returned will include image URLs for generating images from the **render server**. These images are transparent PNGs, as opposed to the JPGs provided by the **image server**.

- `viewNumbers`
- `viewService`
- `viewUrlTemplate`
- `viewUrlShadowTemplate`

These can be processed to generate the URLs you need by adding this function to your code:

```
 /**
* Get the viewUrls for the current design
*
* @param {Object} buildData - the buildData
* @param {Integer} imageWidth - the width value of the image
* @param {Boolean} includeShadow - whether or not to include shadow
* @returns {Array} - an array of urls
*/
getViewUrls(buildData, imageWidth, includeShadow) { // eslint-disable-line class-methods-
use-this
    const viewUrls = [];
    const viewService = buildData.viewService;
    const viewTemplate = (includeShadow ? buildData.viewUrlShadowTemplate :
buildData.viewUrlTemplate);

    if (buildData && buildData.viewNumbers && viewService && viewTemplate) {
        const viewNumbers = privateData.getBuildData().viewNumbers.split(',');
        const baseViewUrl = `${viewService}${viewTemplate}`;

        for (let viewIndex = 0; viewIndex < viewNumbers.length; viewIndex += 1) {
            const viewUrl = `${baseViewUrl.replace(/{VIEW_NUMBER}/,
viewNumbers[viewIndex])}&wid=${imageWidth}`;

            viewUrls.push(viewUrl);
        }
    }

    return viewUrls;
}
```

Methods

applyMyDesign

Description: Loads the design associated with the included designKey into the Builder. The `onProductLoad` bridge method will be called in response to a successful load.

Usage:

```
builderApi.applyMyDesign(1561569431459)
```

clearJerseyCustomization

Description: Clears the customization (un-answers questions) and returns the new build data, for jerseys exclusively.

Usage:

```
builderApi.clearJerseyCustomization()
```

clearMessage

Description: Clears the message in the Builder.

Usage:

```
builderApi.clearMessage()
```

deleteMyDesign

Description: Deletes the myDesign associated with the included designKey from the localStorageCache. See also **deleteMyDesigns**

Usage:

```
builderApi.deleteMyDesign(1561569431459)
```

deleteMyDesigns

Description: Deletes all of the myDesigns from the localStorageCache. See also **deleteMyDesign**

Usage:

```
builderApi.deleteMyDesigns()
```

getAffectedQuestionMap

Description: Return questionsMappingInfo based on the questionId

Usage:

```
 /**
* @returns {Array}
*/
builderApi.getAffectedQuestionMap()
```

getAnswersByCode

Description: Returns answers and their matching questionsId which can then be used to display an answer. Useful for tapping into specific question or answer nodes.

Usage:

```
 /**
* @returns {Array} List of normalized questions
*/
builderApi.getAnswersByCode('jersey')
```

getAvailability

Description: Returns promise to fetch availability information for the current configuration.

Usage:

```
 /**
• @returns {Promise->Array} a promise to return availability data
*/
builderApi.getAvailability()
```

getAvailabilityMessages

Description: Returns promise to fetch availability messages information for the current configuration.

Usage:

```
 /**
• @returns {Promise->Object} a promise to return availability messages including
leadtimeUpperBoundInDays, longCapacityMessage, and shortCapacityMessage
*/
builderApi.getAvailabilityMessages()
```

getBomXML

Description: Returns the Bill Of Materials (BOM) in XML format. Filters out options with blank pidText, meaning, if the user selects only customizable options and enters no text for those options, then there will be no customization returned. Returns a Promise that either:

- Resolves to a metricId, if the builder data is valid

OR

- Rejects with an error message, if the builder data is invalid

Usage:

```
builderApi.getBomXML()
```

getBuild

Description: Returns the current build snapshot. Filters out options with blank pidText, meaning, if the user selects only customizable options and enters no text for those options, then there will be no customization returned. Also, returns the isJerseyCustomized data.

Usage:

```
/**
* @return {Object} The current build snapshot
*/
builderApi.getBuild()
```

getCapacity

Description: Returns promise to fetch capacity information for current or passed-in product configuration.

Usage:

```
/**
* @returns {Promise->Object} a promise to return capacity data
*/
builderApi.getCapacity().then()
// or
builderApi.getCapacity({ country, pathName }).then()
```

getLeadTimeMessage

Description: Returns promise to fetch lead time information for current or passed-in product configuration.

```
/**
* @returns {Promise->Object} a promise to return leadTimeMessage data
*/
builderApi(getLeadTimeMessage)
```

getMarketingComponents

Description: Returns a list of normalized marketing components with nested question keys. The questions index can be used to lookup nested question values.

Usage:

```
/**
```

```
 * @returns {Object} the current prouduct marketing components data
 */
builderApi.getMarketingComponents()
```

getMyDesigns

Description: Gets all of the myDesigns from the localStorageCache. Returns an Array of myDesigns sorted from newest to oldest.

Usage:

```
  /**
 * @returns {Array}
 */
builderApi.getMyDesigns()
```

getProductColorPalette

Description: Returns a list of unique colors for the loaded builder.

Usage:

```
  /**
 * @return {Array} List of selected color answers unique by hex
 */
builderApi.getProductColorPalette()
```

getProductFillColors

Description: Returns a list of unique colors applicable to all marketing components for the loaded Builder

Usage:

```
  /**
 * @returns {Array} List of selected color answers unique by hex
 */
builderApi.getProductFillColors()
```

getQuestionsIndex

Description: Returns a normalized array of questions keyed by their ID. Effectively, this is a flattened tree of all questions contained in the product data with nested questions as keys.

Usage:

```
  /**
```

```
* @returns {Array} List of normalized questions
*/
builderApi.getQuestionsIndex()
```

Example:

```
 {
  pathName:questionId: {
    id: 'pathName:questionId',
    isHidden: false,
    questions: [
      0: 'pathName:questionId:nestedQuestionId'
    ]
  }
}
```

getSelectedColors

Description: Returns a collection of all selected colors from the current build, unique by hex(attribute).

Usage:

```
 /**
* @returns {Array} List of selected color answers unique by hex
*/
builderApi.getSelectedColors()
```

getSelectedColorsPatterns

Description: Returns a collection of all selected colors and patterns from the current build. Colors are unique by hex(attribute) and patterns are unique by src(attribute).

Usage:

```
 /**
* @returns {Array} List of selected color answers unique by hex & pattern type questions
unique by src
*/
builderApi.getSelectedColorsPatterns
```

getSelectedPalette

Description: Returns a list of unique colors for the selected answers of the loaded builder.

Usage:

```
 /**
* @returns {Array} List of selected color answers unique by displayName
*/
builderApi.getSelectedPalette()
```

getSelectedPatterns

Description: Returns a collection of all selected patterns from the current build, unique by src(attribute).

Usage:

```
 /**
* @returns {Array} List of selected pattern answers unique by src
*/
builderApi.getSelectedPatterns
```

getSelectedQuestionAnswerPairs

Description: Returns a collection of all answered questions along with their answer. For most products this will return the majority of questions, as defaults are selected in B16 APIs.

Usage:

```
 /**
* @returns {Array}
*/
builderApi.getSelectedQuestionAnswerPairs
```

getUpCharges

Description: Returns an array of applicable up-charge objects corresponding to the answered questions.

Usage:

```
 /**
* @returns {Object}
*/
builderApi.getUpCharges()
```

Example:

```
 [
   {
     answer: "ANSWER_ID",
     displayName: "Display Name",
```

```
      question: "QUESTION:PATH",
      upCharge: "$0"_
    }
]
```

isJerseyCustomized

Description: Returns information on the jersey customization. Filters out options with blank pidText, meaning, if the user selects only customizable options and enters no text for those options, then the function will return false.

Usage:

```
builderApi.isJerseyCustomized()
```

isPidAllowed

Description: Returns true if the profanity service allowed the string (no stop word was matched). String passed must match records in the capacity service exactly. Only single strings are supported.

Usage:

```
/**
* @returns {Promise->Boolean} a promise to return profanity result
*/
builderApi.isPidAllowed('MYPID')
```

openHighResImageUrl

Description: Returns the high-resolution image URL in a new window.

```
builderApi.openHighResImageUrl()
```

saveBuild (Deprecated)

Description: Persists build data and returns promise to send back a metric ID for that build.

Usage:

```
/**
* @returns {Promise->String}
*/

builderApi.saveBuild()
```

saveDesign

Description: Persists the current build after validating it can be purchased. Filters out options with blank pidText, meaning, if the user selects only customizable options and enters no text for those options, then there will be no customization returned. Returns a Promise that either:

- Resolves to a metricId, if the builder data is valid

OR

- Rejects with an error message, if the builder data is invalid

Usage:

```
 /**
* @returns {Promise->String}
*/

builderApi.saveDesign()
```

saveMyDesign

Description: Saves a myDesign to the localStorageCache. Returns the new Array of myDesigns sorted from newest to oldest.

Usage:

```
 /**
* @returns {Array}
*/
builder.Api.saveMyDesign
```

setAnswer

Description: Returns the current build snapshot after changing the answer to a question in the Builder. Send the full question path, e.g. `"af1High14_July:LTITEM8170:LTITEM12012:LTITEM236023:LTITEM213006"` and the ID of the answer you would like to apply for that question, e.g. `"LTITEM167036"`.

Usage:

```
 /**
* @param {String} questionId
* @param {String} answerId
* @param {String} pidValue
* @returns {Object} Updated build snapshot
*/
builderApi.setAnswer(questionId, answerId, pidValue)
```

setBuild

Description: Allows reloading the Builder with a new product by passing one of the following:

Table 7: Parameters for setBuild Method

Param	Type	Result	Example
`metricId`	String	Build is loaded and applied to the product data.	`setBuild({ metricId: 123456789 });`
`prebuildId`	String	Build is loaded and applied to the product data.	`setBuild({ prebuildId: 123456789 });`
`buildData`	Object	Product for that build is reloaded and has the build data applied to the product.	`setBuild({ buildData: buildData });`

After calling this method the `bridge.onProductLoad` callback is called with the `buildData` for the newly-applied build.

Usage:

```
 /**
* @param {Object} buildData
*/
builderApi.setBuild(buildData)
```

setIsVisible

Description: Informs the Builder whether it is visible on-screen or not. To take effect, `enableAnimationInit: true` needs to be passed in the initialization config options.

Usage:

```
builderApi.setIsVisible(true)
```

setMessage

Description: Causes the Builder to display a message. Takes a message object consisting of the following properties:

Table 8: Properties of Message Object in setMessage

Field	Type	Description
`header`	String	The text displayed at the top of the message.
`content`	String	The text displayed under the header.
`type`	String	The type of message to be displayed, which controls how the message is displayed. Default is ‘COUNTDOWN’. Possible values are: ‘COUNTDOWN’, ‘COUNTDOWN_URGENT’, ‘COUNTDOWN_EXPIRED’

Usage:

```
 /**
* @param {Object} message
*/
builderApi.setMessage(message)
```

Examples:

```
 var countdownMessage = { header: 'Available for', content: '22:14:54:06', type:
'COUNTDOWN' };
builderApi.setMessage(countdownMessage);

var countdownUrgentMessage = { header: 'Available for', content: '00:00:10:06', type:
'COUNTDOWN_URGENT' };
builderApi.setMessage(countdownUrgentMessage);

var countdownExpiredMessage = { header: 'Great choice but...', content: 'This one sold
out fast. Check back soon for availability. Keep customizing, save your design and share
it with friends.', type: 'COUNTDOWN_EXPIRED' };
builderApi.setMessage(countdownExpiredMessage);
```

setSizeAnswer

Description: Given a question ID and answer ID, answer the corresponding question in the build.

Usage:

```
 /**
* @param {String} questionId
* @param {String} answerId
* @param {String} sizeType
* @returns {Object} Updated build snapshot
*/
setSizeAnswer(questionId, answerId, sizeType)
```

setSizeType

Description: Allows updating the Builder's internal state for selected size type, which updates the size, price, and color data when the Builder creates the buildData.

Usage:

```
 /**
* @param {String} sizeType
* @returns {Object} Updated build snapshot
*/
builderApi.setSizeType(sizeType)
```

shareDesign

Description: Persists the current build and returns Promise that resolves to a metricId. Unlike **saveDesign**, with `shareDesign` no validation is done on the build.

showNotification

Description: Causes the Builder to display a notification. It takes a message object consisting of the following properties:

Table 9: Properties of Message Object in showNotification Method

Field	Type	Description
`type`	String	The type of message. Only one message of a given type will be displayed.
`message`	String	The message text.
`level`	String	Optional - Determines the message level. Possible values: 'alert', undefined/null.

Field	Type	Description
`title`	String	Optional - The message title.

Usage:

```
 /**
* @param {Object} message
*/
builderApi.showNotification(message)
```

Examples:

```
 var alert = { message: 'This is an alert with the orange thing', level: 'alert', type:
'some val' };
builderApi.showNotification(alert);

var titled = { message: 'This is just a plain message, but with a title', title: 'Hey
yo!', type: 'some val' };
builderApi.showNotification(titled);

var plain = { message: 'just a plain old message. not that fun, sorry.', type: 'some val'
};
builderApi.showNotification(plain);
```

Contacting the Team

Slack	#nikeid-dev-systems (https://nikedigital.slack.com/archives/C0L8C4UM7)
Confluence Space	NikeiD Systems Home (https://confluence.nike.com/display/NIDS/NikeiD+Systems+Home)
Team Contacts	Jason Mueller, Product Manager

Document Change Log

Summary	Date
Initial publish	05/17/2019
Added new My Designs methods, added new	07/02/2019

Summary	Date
shareDesign, saveDesign methods, marked saveBuild as deprecated	
Added new methods and buildData info for availability	07/30/2019
Added new clearCustomization method	04/09/2020
Added isJerseyCustomized and getBomXML methods, updated descriptions of getBuild and saveDesign	04/24/2020

Next Steps

- **Adding Customization To Your Experience (https://nde-devportal-docs.niketech.com/doc/commerce/customization/use-customization.html)**
- **Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html)**
- **Glossary (https://nde-devportal-docs.niketech.com/doc/commerce/reference/glossary.html)**